

■ Krishi Clinic Lite — Product & Technical Documentation

1. Executive Summary & Product Vision

What is Krishi Clinic Lite?

Krishi Clinic Lite is an intelligent, digital agricultural platform designed to diagnose crop diseases, provide actionable treatment recommendations, and connect smallholder farmers with certified agricultural experts. By combining high-speed AI computer vision with a "human-in-the-loop" verification workflow, the platform delivers immediate safety-first advice while ensuring clinical accuracy.

The Problem

Smallholder farmers in India lose an estimated 30% to 35% of their annual crop yields to preventable pests, weeds, and pathogens. Traditional agricultural extension services are overburdened, and getting a physical visit from a certified agronomist can take days or weeks—during which entire crops can be ruined.

The Solution

Krishi Clinic Lite resolves this gap:

- **Instant AI Diagnosis:** Farmers snap a photo of a diseased leaf and receive an immediate, high-probability analysis.
- **Expert Review Loop:** AI diagnoses are automatically routed to a console for agronomists to audit.
- **Safety First:** Raw AI advice is masked for farmers under a "Pending Review" status to avoid the application of incorrect chemicals. Once an agronomist verifies it, the advice is unlocked.
- **Recovery Tracking:** Farmers take follow-up photos to monitor crop healing progress.

2. The Core User Journey (How It Works)

The product is built around a simple, sequential five-step workflow:

[Farmer Upload] → [AI Analysis & RAG Search] → [Pending Review (Masked)] → [Agronomist Verification] → [Verified Advisory Unlocked]

1. **Farmer Uploads Query:** The farmer snaps a photo of their diseased crop, selects the crop type, notes any observed symptoms (e.g., "yellow spots on lower leaves"), and specifies their regional location.

- 2. **AI & RAG Analysis:** The system immediately analyzes the image and uses text embeddings to search historical cases for similar symptoms verified by experts (Retrieval-Augmented Generation).
- 3. **Graceful Degradation (Safety Gate):** While the AI works, the query goes into the PENDING_REVIEW state. Standard farmers are shown a "Pending Review" banner, protecting them from acting on potential AI hallucinations or errors.
- 4. **Agronomist Review:** A certified agronomist logs into the console, reviews the raw AI recommendation side-by-side with the uploaded photo, overrides any mistakes, and inputs custom advisory notes.
- 5. **Advisory Release:** The status transitions to REVIEWED. The farmer is notified, the verified advisory is unlocked in their native language, and they can download a printable PDF report.
- 6. **Progress Tracking:** The farmer uploads a follow-up image weeks later to document the crop's recovery, building a historical "Before-and-After" case file.

3. Key Tech Stack Summary

- **Frontend Presentation:** Next.js 14 (TypeScript, React, Tailwind CSS, Recharts for analytics, @svg-maps/india for state outbreak indicators).
- **Backend Application API:** FastAPI (Python 3.12, Uvicorn ASGI server, Pydantic V2 schemas).
- **Database Storage:** PostgreSQL 15 + Async SQLAlchemy 2.0 connection pool + Alembic migrations.
- **AI Inference Engine:** Swappable Abstract Base Class (AIProvider) supporting Gemini, OpenAI, Groq, local PyTorch (EfficientNetV2-S), and Mock engines with automatic fallback wrappers.
- **DevOps:** Docker and Docker Compose for single-command builds.

4. API Endpoint Specifications

All endpoints use JSON payloads, consistent error responses, and role-based route middleware protection.

Method	Endpoint	Description	Auth Requirement
GET	/health	Liveness health check + DB check	Open
POST	/api/v1/predictions	Upload crop image for AI diagnosis	Farmer / Admin
GET	/api/v1/predictions	Paginated, filtered list of past cases	Any logged-in user
GET	/api/v1/predictions/{id}	Detailed metadata for a single case	Owner / Agronomist

POST	/api/v1/predictions/{id}/review	Submit expert agronomist verification	Agronomist / Admin
POST	/api/v1/predictions/{id}/followup	Upload recovery tracker image	Owner
GET	/api/v1/analytics/summary	Dashboard analytics (disease trends)	Any logged-in user
GET	/api/v1/predictions/export	Download CSV, JSON, or XML snapshots	Agronomist / Admin
GET	/api/v1/predictions/{id}/pdf	Download PDF advisory report card	Owner / Agronomist

5. Database Schema Design & Migrations

The predictions table utilizes UUID primary keys, TIMESTAMPTZ timestamps, and indexes on created_at and predicted_disease to optimize analytics dashboards performance.

Schema Columns (predictions Table)

Column Name	Type	Description
id	UUID (PK)	Unique identifier, generated via gen_random_uuid()
crop_type	VARCHAR(100)	Crop classification (e.g., Tomato, Wheat, Rice)
image_filename	VARCHAR(255)	Stored image filename, randomized for security
farmer_notes	TEXT	Optional comments entered by the farmer
predicted_disease	VARCHAR(150)	Disease name returned by the AI provider
confidence	FLOAT	Statistical score between 0.0 and 1.0
severity	VARCHAR(50)	High, Medium, or Low severity classification
recommendation	TEXT	Treatment advisory
possible_reasons	TEXT	Pathogen causes or triggers
location	VARCHAR(150)	Farmer's region location
language	VARCHAR(10)	Requested translation language
ai_provider	VARCHAR(50)	Provider that executed the diagnosis
status	VARCHAR(50)	PENDING_REVIEW or REVIEWED status
farmer_id	UUID (FK)	Reference to the uploading User
agronomist_id	UUID (FK)	Reference to the reviewing Agronomist
agronomist_predicted_disease	VARCHAR(150)	Agronomist override disease name
agronomist_severity	VARCHAR(50)	Agronomist override severity
agronomist_review	TEXT	Expert comments
reviewed_at	TIMESTAMPTZ	Agronomist review timestamp
created_at	TIMESTAMPTZ	Timestamp of initial upload

after_image_filename	VARCHAR(255)	Follow-up recovery image filename
after_notes	TEXT	Follow-up progress comments
after_uploaded_at	TIMESTAMPTZ	Follow-up upload timestamp
notes_embedding	Vector(1536)	Text embedding used for RAG matching

6. Swappable AI Interface & Local PyTorch Inference

Decoupled AI Architecture

AI operations are hidden behind the abstract `AIProvider` contract. Swapping providers requires updating the `AI_PROVIDER` environment variable, requiring **zero code modifications** in API routing or service layers.

5 swappable AI Engines

1. **Mock Provider** (`mock`): Returns deterministic response objects depending on crop parameters. No API keys needed; ideal for tests and dev runs.
2. **Google Gemini** (`gemini`): Connects to the `google-genai` SDK using `gemini-1.5-flash` or `gemini-2.0-flash`. Enforces structured JSON schema outputs.
3. **Groq API** (`groq`): Invokes Llama-3 models via Groq REST endpoints.
4. **OpenAI SDK** (`openai`): Integrates GPT-4o-mini structured schema APIs.
5. **Local PyTorch** (`local`): Edge inference using a fine-tuned **EfficientNetV2-S** model. Automatically caches model weights from the Hugging Face Hub, allowing completely offline local inference.

Fallback Provider Wrapper

Includes a wrapper fallback provider. If Gemini or OpenAI fails (due to network error, invalid API key, or rate limits), the fallback wrapper catches the exception and degrades gracefully to Mock data. It saves `ai_provider: "gemini (fallback to mock)"` to the database, ensuring system availability.

RAG (Retrieval-Augmented Generation)

Before querying LLMs, the service vectorizes the crop type and notes, searches the database for similar verified agronomist cases, and injects the top 2 matching case details directly into the LLM system prompt. This drastically improves diagnosis quality.

7. Advanced Application Features

JWT Expert Review Workflow

- **Roles:** FARMER, AGRONOMIST, ADMIN (passwords hashed via bcrypt).
- **Masking:** Queries uploaded by farmers default to the PENDING_REVIEW state. Standard farmers receive a masked response ("Pending expert review") to avoid acting on inaccurate raw AI predictions.
- **Agronomist Portal:** Agronomists can log in, view raw AI data, verify the disease, customize recommendations, and confirm status as REVIEWED, unlocking the advisory.

PDF Report Card Generation

Provides GET /predictions/{id}/pdf generating custom PDF advisory sheets using ReportLab, complete with crop metadata, comparison before-after photographs, and verified agronomist treatment details.

Outbreak India Heatmap

Frontend features an interactive vector map of India's states utilizing the @svg-maps/india package. It dynamically reads location outbreaks from the analytics endpoint, highlighting state risk zones in red/green with smooth transitions.

8. Local Setup Instructions

Docker Compose Quick-Start

To bring up Next.js frontend, FastAPI backend, and PostgreSQL database with a single command:

1. Copy Environment File:

```
cp .env.example .env
```

(Set GEMINI_API_KEY or OPENAI_API_KEY to run real models; otherwise it defaults to the mock fallback).

2. Build and Start:

```
docker compose up --build
```

- **Frontend:** http://localhost:3000
 - **Backend API:** http://localhost:8000
 - **Swagger Documentation:** http://localhost:8000/docs
-

9. Quality Assurance & Tests

Contains **21 unit and integration tests** verifying:

- FastAPI endpoints execution and health routines.
- Image size and MIME validation logic.
- RAG cosine similarity calculations.
- AI provider interface contracts and fallback wrappers.
- Role-based agronomist review workflow.

Run the test suite:

```
cd backend .venv/bin/pytest tests/ -v
```

10. Engineering Decisions & Rationale

1. **Layered Service Architecture:** Separated endpoints from business logic. Allows changing storage, AI models, and database components with zero impact on web routes.
2. **Spooled File Uploads:** Used `UploadFile` spooled file descriptors instead of raw bytes. This streams uploads to temporary disk files, preventing Out-Of-Memory (OOM) failures under concurrent traffic.
3. **Local Image Storage with Provider Abstraction:** Stored image files in the local container volumes using `secrets.token_hex(16)` filenames (prevents name collisions and path traversal attacks). Kept behind a `StorageProvider ABC`, making future S3/GCS integrations highly decoupled.
4. **Alembic Database Versioning:** Avoided static SQL files by versioning SQL tables through Alembic migrations.

11. Reflection Questions

- **Hardest challenge:** Designing the swappable AI engine and Structured Outputs contract. Solved by defining standard Pydantic models for inputs and outputs, ensuring LLM engines return valid, parseable JSON via JSON schema configuration.
- **Biggest trade-off:** Storing crop images on the local disk instead of S3. Solved by placing storage behind an interface, allowing future AWS integrations to be completed with minimal code edits.
- **Biggest learning:** The elegance of FastAPI dependency injection (`Depends()`), which simplified injecting mock sessions, mock AI providers, and test storages during pytest suites.